AGENTIC QA APPLICATION

Architecture and Code Handbook

LangGraph + LangChain + Pydantic + pytest + Playwright

Project snapshot: 2 September 2026

Current implementation: Requirement Analyzer + V1 Repository Retrieval

How to use this handbook

This document is a learning and architecture reference for the Agentic QA application we have been building. It explains both the code that exists now and the architecture we are intentionally evolving toward. The goal is that you can read it away from the chat, return to the repository, and understand why each file, model, abstraction, node, and test exists.

Status convention

IMPLEMENTED means the behavior exists in the current application. PLACEHOLDER means the node is present in the graph but does not yet perform its final responsibility. PLANNED means a V2/V3 capability described here for the next implementation phases.

What is included

- The product goal and testing philosophy.
- The current repository organization and separation between the AI application and sample automation framework.
- LangGraph state, nodes, routing, dependency injection, Protocols, and Pydantic contracts.
- Requirement analysis with structured LLM output and deterministic evaluation.
- V1 repository retrieval: query construction, file discovery, artifact classification, scoring, snippets, and limitations.
- The sample pytest + Playwright Page Object Model framework used as retrieval corpus.
- Unit tests, fakes, typing decisions, and the mypy issues we solved.
- Real V1 retrieval runs and what they taught us.
- The V2 AST structure-aware retrieval plan and V3 semantic RAG + exact symbol lookup plan.
- Future Playwright MCP exploration, TestDesign, code generation, execution, failure analysis, and repair policy.

What this document is not

It is not claiming that the full self-repairing testing agent already exists. Today, the strongest implemented path is requirement analysis followed by V1 repository retrieval. Later graph nodes are intentionally present as placeholders so we can evolve one controlled workflow rather than starting with many agents and uncontrolled loops.

Contents

1. Product goal and engineering philosophy
2. Current system at a glance
3. Repository and package layout
4. Core architectural concepts
5. Domain models and LangGraph state
6. Requirement Analyzer - implementation and reasoning
7. Requirement Analyzer evaluation framework
8. Sample pytest + Playwright POM framework
9. Repository Retrieval V1 - design and implementation
10. LangGraph nodes and workflow wiring
11. Composition root, configuration, and runtime execution
12. Unit testing, fakes, and typing strategy
13. Real V1 retrieval experiments and findings
14. Why V1 is now frozen
15. Repository Retrieval V2 - AST structure-aware design
16. Repository Retrieval V3 - semantic RAG + exact symbols
17. Future Playwright MCP live exploration
18. Future TestDesign and code generation
19. Future execution, result analysis, and repair
20. Evaluation and observability strategy
21. Security, configuration, and maintainability
22. Development roadmap
23. Glossary
24. Appendix A. Consolidated current reference code
25. Appendix B. Debugging lessons learned

1. Product goal and engineering philosophy

The application is an agentic QA automation system. Its purpose is not to generate isolated Playwright snippets from a prompt. It is intended to behave like a controlled QA engineering workflow that gathers evidence before it designs or changes automation.

Target end-to-end workflow

```
Requirement
    -> Analyze
    -> Retrieve repository context
    -> Inspect exact symbols
    -> Explore live UI
    -> Design test
    -> Validate design
    -> Generate maintainable test
    -> Execute
    -> Analyze result
    -> Repair or Finish
```

Why a graph instead of one giant prompt

- Each node has one responsibility and a measurable output.
- State transitions are observable and testable.
- We can retry only the node that needs another attempt.
- Different tools can be attached to different responsibilities.
- We can classify failures before changing code.
- We avoid hidden, uncontrolled loops where an LLM repeatedly rewrites tests until they pass.

Testing scope

The system is focused on generating and maintaining E2E/UI tests and integration/API tests. It should choose the appropriate level rather than automate everything through a browser. UI/E2E is reserved for important user journeys and browser-specific behavior; integration/API tests should be preferred when they provide faster, more stable coverage for the same business rule.

Permanent automation vs exploration

Capability	Purpose	Permanent regression?
Playwright MCP	Explore the live application, inspect accessible roles/names, verify behavior, validate assumptions.	No
Python Playwright + pytest	Maintainable regression tests in repository.	Yes
RAG	Find semantically similar existing tests, Page Objects, fixtures, guidelines.	Knowledge source
AST / symbol index	Answer exact code questions: method existence, signatures, fixture definitions, imports.	Knowledge source
LangGraph	Orchestrate nodes, state, routing, retries, conditions.	Application control plane

2. Current system at a glance

The current application has two genuinely implemented capabilities: structured requirement analysis and V1 repository retrieval. The later nodes remain intentionally simple placeholders.

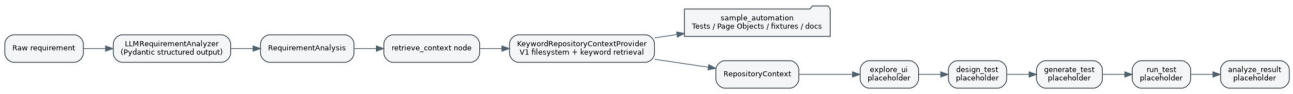


Figure 1. Current workflow and implementation status.

Implemented now

- Pydantic RequirementAnalysis schema.
- RequirementAnalyzer Protocol and LLMRequirementAnalyzer implementation.
- ChatOpenAI structured output through Pydantic.
- LangGraph QAState and graph construction.
- RepositoryContextProvider Protocol.
- KeywordRepositoryContextProvider V1.
- Typed RepositoryContext / RetrievedContextItem models.
- Sample pytest + Playwright POM framework used as retrieval corpus.
- Unit tests for nodes, workflow, and V1 retrieval.
- Deterministic requirement-analyzer evaluation corpus and baseline runs.

Placeholders now

- explore_ui
- design_test
- generate_test
- run_test
- analyze_result

Key engineering choice

We are building one reliable workflow first. Multiple specialized agents can be introduced later only when each role has a clear responsibility and measurable value.

3. Repository and package layout

```
agentic-qa/
|-- src/
|   `-- agentic_qa/
|       |-- config.py
|       |-- main.py
|       |-- models.py
|       |-- graph/
|       |   |-- state.py
|       |   |-- nodes.py
|       |   `-- workflow.py
|       |-- requirements/
|       |   |-- analyzer.py
|       |   `-- llm_analyzer.py
|       |-- retrieval/
|       |   |-- models.py
|       |   |-- provider.py
|       |   `-- keyword_provider.py
|       `-- evals/
|           `-- requirements/
|               `-- cases.py
|
|-- sample_automation/
|   |-- pages/
|   |   |-- base_page.py
|   |   `-- todo_page.py
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_todo_creation.py
|   |   `-- test_todo_completion.py
|   `-- README.md
|
|-- tests/
|   `-- unit/
|       |-- fakes.py
|       |-- test_analyze_requirement.py
|       |-- test_keyword_repository_provider.py
|       `-- test_workflow.py
|
|-- scripts/
|   `-- evaluate_requirement_analyzer.py
|-- evaluation_results/
|-- pyproject.toml
`-- .env
```

Why sample_automation is separate

There are two different systems in the repository: the AI application itself and the automation code that the AI application is supposed to inspect. Keeping sample_automation separate prevents our agent unit tests from being confused with regression automation that the retriever should learn from.

```
tests/
    -> tests the AI application itself

sample_automation/tests/
    -> represents the regression framework that the AI application inspects
```

4. Core architectural concepts

4.1 LangGraph state

QAState is the shared structured memory of one workflow execution. It answers: What does the workflow know right now? Nodes read the fields they need and return partial state updates. LangGraph merges those updates into the graph state.

```
Input state
{
    "requirement": "User should be able to complete an existing todo."
}

After analyzer node
{
    "requirement": "...",
    "requirement_analysis": RequirementAnalysis(...)
}

After retrieval node
{
    "requirement": "...",
    "requirement_analysis": RequirementAnalysis(...),
    "repository_context": RepositoryContext(...)
}
```

4.2 Nodes return partial updates

A node does not need to reconstruct the whole QAState. The analyzer node returns only requirement_analysis; the retrieval node returns only repository_context. LangGraph merges these updates. This is why typing a node return as a complete QAState would be misleading.

4.3 Pydantic domain contracts

Pydantic models give runtime validation and machine-readable schemas. They are especially important at LLM boundaries because they force the model output into a predictable shape. However, schema correctness is not semantic correctness: an LLM can produce valid JSON with a wrong interpretation.

4.4 Dependency inversion through Protocol

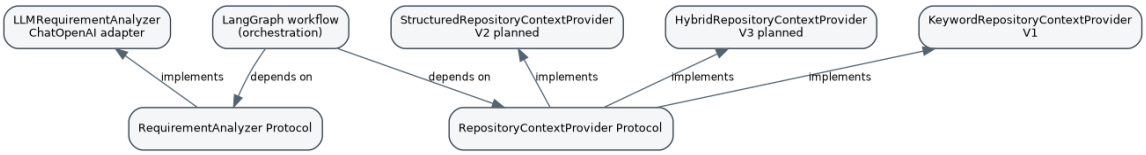


Figure 2. The graph depends on interfaces, not concrete infrastructure.

The graph does not create ChatOpenAI or read files directly. Instead, it depends on RequirementAnalyzer and RepositoryContextProvider protocols. This lets unit tests inject fakes and lets V1, V2, and V3 retrieval implementations change without rewriting the graph.

4.5 Composition root

main.py is where concrete infrastructure is created and injected into the workflow. This keeps configuration and external dependencies at the application boundary.

5. Domain models and LangGraph state

5.1 RequirementAnalysis

```
class InteractionSurface(StrEnum):
    UI = "ui"
    API = "api"
    SYSTEM = "system"
    UNKNOWN = "unknown"


class RequirementAnalysis(BaseModel):
    feature: str
    operation: str
    objective: str
    actors: list[str]
    conditions: list[str]
    expected_outcomes: list[str]
    ambiguities: list[str]
    interaction_surface: InteractionSurface
```

Field	Meaning	Important rule
feature	Business/functional area.	Do not invent implementation details.
operation	The requested behavior/action.	Preserve the requirement meaning.
objective	Short verification objective.	Still analysis, not test steps.
actors	Distinct active participants.	Do not treat ordinary data entities as actors.
conditions	Explicit conditions/preconditions/constraints.	Only supported facts.
expected_outcomes	Observable expected behavior.	Do not invent additional outcomes.
ambiguities	Material missing/unclear information.	Capture only things that can change design or interpretation.
interaction_surface	ui/api/system/unknown.	Use unknown when the requirement does not make the surface clear.

5.2 RepositoryContext

```
class RepositoryArtifactType(StrEnum):
    TEST = "test"
    PAGE_OBJECT = "page_object"
    FIXTURE = "fixture"
    GUIDELINE = "guideline"
    OTHER = "other"


class RetrievedContextItem(BaseModel):
    path: str
    artifact_type: RepositoryArtifactType
    score: float
    matched_terms: list[str] = Field(default_factory=list)
    snippet: str


class RepositoryContext(BaseModel):
    query_terms: list[str] = Field(default_factory=list)
    items: list[RetrievedContextItem] = Field(default_factory=list)
```

5.3 QAState

```
class QAState(TypedDict, total=False):
```

```
    requirement: Required[str]
    requirement_analysis: RequirementAnalysis
    repository_context: RepositoryContext
    exploration_findings: list[str]
    test_design: TestDesign
    generated_code: str
    execution_result: TestExecutionResult
    result_analysis: ResultAnalysis
```

Why TypedDict plus Pydantic?

TypedDict describes the evolving graph-state shape with optional-by-stage fields. Pydantic models validate the important structured domain values placed inside that state.

6. Requirement Analyzer - implementation and reasoning

6.1 Responsibility boundary

The analyzer converts unstructured QA intent into structured understanding. It does not search the repository, explore the browser, design test steps, generate code, or execute pytest. That narrow responsibility keeps the node easier to evaluate.

```
Raw requirement
    -> RequirementAnalyzer
    -> RequirementAnalysis
```

6.2 Protocol

```
class RequirementAnalyzer(Protocol):
    def analyze(self, requirement: str) -> RequirementAnalysis:
        ...
```

6.3 LLM implementation

```
class LLMRequirementAnalyzer:
    def __init__(self, model: BaseChatModel) -> None:
        self._model = model.with_structured_output(
            RequirementAnalysis,
            method="json_schema",
            strict=True,
        )

    def analyze(self, requirement: str) -> RequirementAnalysis:
        messages = [
            SystemMessage(content=SYSTEM_PROMPT),
            HumanMessage(
                content=f"Analyze the following test requirement:\n\n{requirement}"
            ),
        ]

        result = self._model.invoke(messages)

        if not isinstance(result, RequirementAnalysis):
            raise TypeError("Expected RequirementAnalysis")

        return result
```

6.4 Prompt rules

- Extract only information supported by the requirement.
- Do not design test steps.
- Do not generate automation code.
- Do not assume UI or API unless the requirement makes it clear.
- Use interaction_surface=unknown when the surface cannot be determined.
- Capture material ambiguities only.
- Do not invent missing behavior.

6.5 Why interaction_surface is not test_level

interaction_surface records what the requirement explicitly exposes. It is not a downstream testing strategy decision. A requirement can be UI-exposed but still have lower-level integration coverage; conversely, a business requirement may have unknown surface until repository and product context are retrieved.

```
interaction_surface = what interface is explicitly indicated?
```

```
test level = how should we test it?

These are different decisions.
```

6.6 A real semantic error discovered

For the requirement "User should be able to create a new todo", the analyzer produced "Create a new todo list". The JSON was structurally valid, but the meaning changed from a todo item to a todo list. This is an important example of why Pydantic validation cannot replace semantic evaluation.

7. Requirement Analyzer evaluation framework

The analyzer is evaluated separately from the graph. This prevents downstream failures from being confused with requirement-understanding failures.

7.1 Deterministic case model

```
class RequirementEvaluationCase(BaseModel):
    name: str
    requirement: str
    expected_surface: InteractionSurface

    required_condition_terms: list[str] = Field(default_factory=list)
    required_condition_alternatives: list[list[str]] = Field(default_factory=list)

    required_outcome_terms: list[str] = Field(default_factory=list)
    required_outcome_alternatives: list[list[str]] = Field(default_factory=list)

    require_ambiguities: bool | None = None
```

7.2 Terms vs alternatives

Stable facts such as HTTP status codes or important UI labels are checked as exact-ish terms. Natural-language concepts use alternative groups so the evaluator does not fail merely because the model said "already exists" instead of "existing".

```
required_condition_alternatives=[
    ["existing", "exists", "already exists", "available for deletion"],
]

# Meaning:
# at least one phrase from the inner list must appear.
```

7.3 Baseline result

The current deterministic regression dataset reached 100/100 configured checks for the gpt-5.6-terra baseline after evaluator false negatives were corrected. That must be interpreted as a 100% check score on the configured dataset, not proof of 100% semantic model accuracy.

> **Important**
>
> Freeze successful evaluation cases as regression cases. Do not repeatedly add accepted phrases until every new model output passes. New cases should be added when real downstream behavior reveals a meaningful analyzer defect.

7.4 Known analyzer weakness

The actors field occasionally treats domain entities such as Component as actors. That is a semantic weakness we have observed, but it is lower priority than preserving conditions, outcomes, and interaction surface. We intentionally avoided prompt overfitting at this stage.

8. Sample pytest + Playwright Page Object Model framework

We created a small real automation corpus before implementing retrieval. Otherwise retrieve_context would be designed against an imaginary repository. The sample framework targets the TodoMVC demonstration application and uses pytest + Playwright synchronous API + Page Objects + fixtures.

8.1 Architecture

```
Tests
    -> Fixtures
    -> Page Objects
    -> Playwright
    -> Application
```

8.2 BasePage

```
from playwright.sync_api import Page


class BasePage:
    def __init__(self, page: Page, base_url: str) -> None:
        self.page = page
        self.base_url = base_url

    def open(self) -> None:
        self.page.goto(self.base_url)
```

8.3 TodoPage

```
from playwright.sync_api import Locator, Page

from sample_automation.pages.base_page import BasePage


class TodoPage(BasePage):
    def __init__(self, page: Page, base_url: str) -> None:
        super().__init__(page=page, base_url=base_url)

        self.new_todo_input = page.get_by_role(
            "textbox",
            name="What needs to be done?",
        )
        self.todo_items = page.get_by_role("listitem")
        self.completed_filter = page.get_by_role("link", name="Completed")
        self.active_filter = page.get_by_role("link", name="Active")
        self.all_filter = page.get_by_role("link", name="All")

    def add_todo(self, title: str) -> None:
        self.new_todo_input.fill(title)
        self.new_todo_input.press("Enter")

    def todo_item(self, title: str) -> Locator:
        return self.todo_items.filter(has_text=title)

    def todo_checkbox(self, title: str) -> Locator:
        return self.todo_item(title).get_by_role(
            "checkbox",
            name="Toggle Todo",
        )

    def complete_todo(self, title: str) -> None:
        self.todo_checkbox(title).check()

    def show_completed(self) -> None:
        self.completed_filter.click()

    def show_active(self) -> None:
        self.active_filter.click()
```

```
    def show_all(self) -> None:
        self.all_filter.click()
```

8.4 Fixtures

```
@pytest.fixture
def todo_app_url() -> str:
    return os.getenv(
        "TODO_APP_URL",
        "https://demo.playwright.dev/todomvc/",
    )


@pytest.fixture
def todo_page(page: Page, todo_app_url: str) -> TodoPage:
    todo_page = TodoPage(page=page, base_url=todo_app_url)
    todo_page.open()
    return todo_page


@pytest.fixture
def existing_todo(todo_page: TodoPage) -> str:
    title = "Buy groceries"
    todo_page.add_todo(title)
    return title
```

8.5 Tests

```
def test_user_can_complete_todo(
    todo_page: TodoPage,
    existing_todo: str,
) -> None:
    todo_page.complete_todo(existing_todo)

    expect(
        todo_page.todo_checkbox(existing_todo)
    ).to_be_checked()


def test_completed_todo_appears_in_completed_filter(
    todo_page: TodoPage,
    existing_todo: str,
) -> None:
    todo_page.complete_todo(existing_todo)
    todo_page.show_completed()

    expect(
        todo_page.todo_item(existing_todo)
    ).to_be_visible()
```

8.6 Why this corpus matters to the AI

It creates realistic repository knowledge: an existing test pattern, a Page Object method, a reusable existing_todo fixture, and explicit framework conventions in README.md. The agent can later retrieve and reuse these instead of inventing selectors, setup, or duplicate helper methods.

9. Repository Retrieval V1 - design and implementation

9.1 Responsibility

V1 converts RequirementAnalysis into a small set of repository files and snippets that appear lexically relevant. It intentionally uses no embeddings and no AST. The purpose is to establish the retrieval contract and observe the limitations of naive keyword matching.

9.2 Protocol

```
class RepositoryContextProvider(Protocol):
    def retrieve(
        self,
        analysis: RequirementAnalysis,
    ) -> RepositoryContext:
        ...
```

9.3 V1 pipeline

```
RequirementAnalysis
    -> build query terms
    -> walk supported repository files
    -> ignore caches / .git / .venv
    -> classify file type
    -> score lexical matches
    -> extract first matching snippet
    -> sort descending
    -> return top_k RepositoryContext items
```

9.4 Query-term construction

```
values = [
    analysis.feature,
    analysis.operation,
    *analysis.conditions,
    *analysis.expected_outcomes,
]

text = " ".join(values)
words = re.findall(r"[A-Za-z0-9_]+", text.lower())

terms = {
    word
    for word in words
    if len(word) >= 3
    and word not in self.STOP_WORDS
}
```

The objective and ambiguities are deliberately not included in query construction because objective largely repeats the requirement while ambiguities often contain generic phrases that can add retrieval noise.

9.5 File discovery and ignore policy

```
SUPPORTED_SUFFIXES = {".py", ".md"}

IGNORED_DIRECTORIES = {
    ".git",
    ".venv",
    "__pycache__",
    ".pytest_cache",
    ".mypy_cache",
    ".ruff_cache",
}
```

9.6 File classification

Rule	Artifact type
conftest.py	FIXTURE
file under tests/ and name starts test_	TEST
file under pages/	PAGE_OBJECT
.md	GUIDELINE
anything else	OTHER

9.7 Scoring

```
for term in query_terms:
    term_score = 0.0

    if term in path_text:
        term_score += 5.0

    occurrences = content_text.count(term)

    if occurrences > 0:
        term_score += min(float(occurrences), 3.0)

    if term_score > 0:
        matched_terms.append(term)
        score += term_score
```

Path matches receive a strong weight because filenames such as test_todo_completion.py often carry useful intent. Content frequency is capped so one repeated generic term does not dominate indefinitely.

9.8 Snippet extraction

V1 returns roughly five lines before and after the first line containing any matched term. This is intentionally simple, but the real experiments show why it is insufficient: the first lexical match is often not the most relevant method.

9.9 Why not retrieve whole files

Large repositories can contain hundreds of tests and Page Objects. Returning full files would rapidly inflate graph state and LLM prompts. Even V1 therefore returns compact snippets; V2 will improve this by indexing structural code units rather than clipping text around the first match.

10. LangGraph nodes and workflow wiring

10.1 QANode Protocol and the mypy problem

We originally typed node factories as Callable[[QAState], dict[str, ...]]. Current LangGraph typing rejected those factories in add_node even though the functions were valid at runtime. The clean solution was a small structural Protocol that matches the callable shape without importing LangGraph private StateNode internals.

```
class QANode(Protocol):
    def __call__(
        self,
        state: QAState,
    ) -> Any:
        ...
```

10.2 Analyzer node factory

```
def create_analyze_requirement_node(
    analyzer: RequirementAnalyzer,
) -> QANode:
    def analyze_requirement(
        state: QAState,
    ) -> dict[str, RequirementAnalysis]:
        analysis = analyzer.analyze(state["requirement"])

        return {
            "requirement_analysis": analysis,
        }

    return analyze_requirement
```

10.3 Retrieval node factory

```
def create_retrieve_context_node(
    provider: RepositoryContextProvider,
) -> QANode:
    def retrieve_context(
        state: QAState,
    ) -> dict[str, RepositoryContext]:
        context = provider.retrieve(
            state["requirement_analysis"]
        )

        return {
            "repository_context": context,
        }

    return retrieve_context
```

10.4 explore_ui placeholder

```
def explore_ui(
    state: QAState,
) -> dict[str, list[str]]:
    analysis = state["requirement_analysis"]

    if analysis.interaction_surface != InteractionSurface.UI:
        return {"exploration_findings": []}

    return {
        "exploration_findings": [
            "UI exploration is not connected yet.",
        ]
    }
```

For now, explicit API and unknown requirements skip UI exploration. Later, unknown can be resolved using repository evidence and application knowledge before routing.

10.5 Workflow construction

```
def build_workflow(
    requirement_analyzer: RequirementAnalyzer,
    repository_context_provider: RepositoryContextProvider,
) -> CompiledStateGraph[QAState, None, QAState, QAState]:
    graph = StateGraph(QAState)

    graph.add_node(
        "analyze_requirement",
        create_analyze_requirement_node(requirement_analyzer),
    )
    graph.add_node(
        "retrieve_context",
        create_retrieve_context_node(repository_context_provider),
    )
    graph.add_node("explore_ui", explore_ui)
    graph.add_node("design_test", design_test)
    graph.add_node("generate_test", generate_test)
    graph.add_node("run_test", run_test)
    graph.add_node("analyze_result", analyze_result)

    graph.add_edge(START, "analyze_requirement")
    graph.add_edge("analyze_requirement", "retrieve_context")
    graph.add_edge("retrieve_context", "explore_ui")
    graph.add_edge("explore_ui", "design_test")
    graph.add_edge("design_test", "generate_test")
    graph.add_edge("generate_test", "run_test")
    graph.add_edge("run_test", "analyze_result")
    graph.add_edge("analyze_result", END)

    return graph.compile()
```

10.6 Why factories are useful

A factory closes over an injected dependency and returns a normal LangGraph node. This keeps the actual node signature state-only while still allowing different implementations to be injected when the workflow is constructed.

11. Composition root, configuration, and runtime execution

11.1 main.py responsibilities

- Read validated settings.
- Construct ChatOpenAI.
- Wrap it in LLMRequirementAnalyzer.
- Construct the concrete V1 repository provider.
- Inject both into build_workflow.
- Invoke the compiled graph with the raw requirement.

11.2 SecretStr typing lesson

ChatOpenAI typing expects a SecretStr-compatible API key. Calling get_secret_value() converts it to plain str and caused a mypy error. The cleaner approach is to keep the API key as Pydantic SecretStr and pass it directly after checking it is not None.

```
if settings.openai_api_key is None:
    raise RuntimeError("OPENAI_API_KEY is not configured.")

model = ChatOpenAI(
    model=settings.llm_model,
    api_key=settings.openai_api_key,
)
```

11.3 Workflow return type

mypy also required create_workflow to have an explicit return type. Rather than using Any, we use the compiled LangGraph type so IDEs and type checking understand invoke/stream behavior.

```
def create_workflow() -> CompiledStateGraph[
    QAState,
    None,
    QAState,
    QAState,
]:
    ...
```

11.4 Target repository configuration

```
TARGET_REPOSITORY=sample_automation
```

The retriever should depend on this configured path, not on hard-coded sample_automation knowledge. Later the same provider contract can point to a real regression repository without changing LangGraph.

12. Unit testing, fakes, and typing strategy

12.1 Why fakes

Unit tests should not require an OpenAI API key, network access, or real filesystem repository unless the unit under test is repository retrieval itself. We inject deterministic fakes through the same protocols used in production.

12.2 FakeRequirementAnalyzer

```
class FakeRequirementAnalyzer:
    def __init__(self) -> None:
        self.result = RequirementAnalysis(
            feature="Component management",
            operation="Delete an existing component",
            objective="Verify that an existing component can be deleted.",
            actors=["User"],
            conditions=["A component already exists."],
            expected_outcomes=["The component can be deleted."],
            ambiguities=["The interaction surface is not specified."],
            interaction_surface=InteractionSurface.UNKNOWN,
        )

    def analyze(self, requirement: str) -> RequirementAnalysis:
        return self.result
```

12.3 Better node assertion

The node test should verify that the node places the analyzer result into state. It should not duplicate every semantic string from the fake because those details belong to analyzer tests/evaluation.

```
result = node({
    "requirement": "User should be able to delete an existing component."
})

assert result["requirement_analysis"] == analyzer.result
```

12.4 V1 retriever tests

- Relevant todo Page Object and completion test are retrieved.
- Unrelated login test is not retrieved.
- Page Object, test, and fixture artifact classification works.
- Missing repository path raises FileNotFoundError.

12.5 Schema-evolution lesson

When RequirementAnalysis changed from needs_ui_exploration to interaction_surface plus richer fields, the production analyzer was updated but the FakeRequirementAnalyzer still created the old schema. Pydantic caught this at runtime. Whenever a domain contract evolves, update production adapters, fakes, and assertions together.

13. Real V1 retrieval experiments and findings

We ran three real requirements through the actual LLM analyzer and V1 retriever. These runs are more valuable than hypothetical discussion because they expose exactly where simple lexical retrieval succeeds and fails.

13.1 Case A - complete an existing todo

```
Requirement:
User should be able to complete an existing todo.

Analyzer query terms:
already, complete, existing, exists, management, todo

Top retrieval results:
14  tests/test_todo_completion.py
11  pages/todo_page.py
10  tests/test_todo_creation.py
 4  tests/conftest.py
 2  README.md
```

This is a good V1 success case. The correct completion test is first, the Page Object is second, and the fixture/guidelines are found. However, test_todo_creation.py also scores highly because generic words such as todo and existing occur there.

13.2 Case B - create a new todo

```
Requirement:
User should be able to create a new todo.

Analyzer semantic issue:
operation = "Create a new todo list"
objective = "Verify that a user can create a new todo list."

Top retrieval results:
12  pages/todo_page.py
11  tests/test_todo_creation.py
 8  tests/test_todo_completion.py
 3  tests/conftest.py
 2  README.md
```

Two separate lessons appear here. First, the analyzer changed the concept from a todo item to a todo list - a semantic defect that should become a new analyzer evaluation case. Second, file-level scoring ranks the whole Page Object above the existing creation test, even though the test is the more directly reusable scenario.

13.3 Case C - mark an existing task as done

```
Requirement:
User should be able to mark an existing task as done.

Repository vocabulary:
todo, complete, completed

Requirement vocabulary:
task, mark, done

Top retrieval results:
3  tests/test_todo_completion.py   (matched: existing)
2  README.md                       (matched: existing)
2  tests/test_todo_creation.py     (matched: existing)
1  pages/todo_page.py              (matched: done)
1  tests/conftest.py               (matched: existing)
```

Critical observation

V1 placed the correct completion test first, but largely for the wrong reason: it matched the generic word existing. It did not understand task ~= todo or done ~= completed.

13.4 Accidental lexical match

todo_page.py matched the word done because the input placeholder is "What needs to be done?". That line is unrelated to completing a todo. The retriever therefore returned the constructor area instead of the relevant complete_todo method. This demonstrates both a ranking problem and a snippet-selection problem.

13.5 V1 capability summary

Capability	V1 result
Find same-vocabulary existing test	Good
Find Page Object file	Good
Find fixture	Reasonable
Find guidelines	Good
Rank exact operation	Weak
Understand synonyms	No
Find relevant method	No
Distinguish semantic vs accidental word match	No
Return best code snippet	No
Answer exact symbol questions	No

14. Why V1 is now frozen

We could keep improving V1 with stemming, synonym dictionaries, TF-IDF, more elaborate weights, and better text windows. That would teach us less than moving to the next architectural layer. V1 has already fulfilled its purpose: establish the retrieval interface, prove graph integration, and reveal concrete limitations.

Decision

Do not spend significant time tuning V1. Keep the three real todo requirements as regression examples and use them to compare V2 and V3.

Regression examples to preserve

Case	Requirement	What it tests
A	Complete an existing todo	Lexical + structural match
B	Create a new todo	Creation retrieval plus analyzer concept preservation
C	Mark an existing task as done	Semantic vocabulary mismatch

15. Repository Retrieval V2 - AST structure-aware design

V2 keeps the same RepositoryContextProvider boundary but changes the retrieval unit from file to code structure. It still uses keyword-style ranking initially; the improvement is that it understands classes, methods, tests, and fixtures as independent objects.

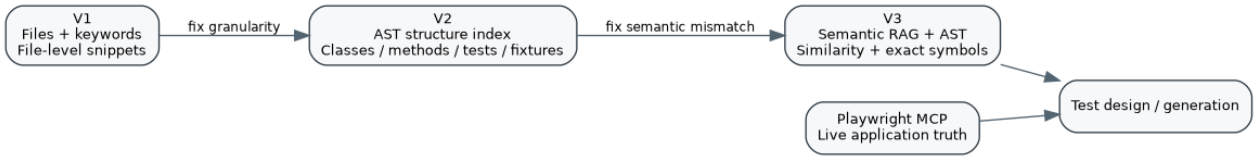


Figure 3. Retrieval evolution from file-level keywords to semantic + exact code intelligence.

15.1 Why AST

Python abstract syntax trees provide deterministic structure without requiring an LLM. For a Page Object with 50 methods, AST lets us index complete_todo separately from add_todo and delete_todo instead of retrieving the entire file.

15.2 Planned symbol model

```
class SymbolType(StrEnum):
    CLASS = "class"
    METHOD = "method"
    FUNCTION = "function"
    TEST = "test"
    FIXTURE = "fixture"


class RepositorySymbol(BaseModel):
    name: str
    symbol_type: SymbolType
    path: str
    parent: str | None = None
    source: str
    start_line: int
    end_line: int
```

15.3 Planned repository index

```
sample_automation/pages/todo_page.py
    CLASS   TodoPage
    METHOD  TodoPage.add_todo
    METHOD  TodoPage.todo_item
    METHOD  TodoPage.todo_checkbox
    METHOD  TodoPage.complete_todo
    METHOD  TodoPage.show_completed
    METHOD  TodoPage.show_active

sample_automation/tests/conftest.py
    FIXTURE todo_page
    FIXTURE existing_todo

sample_automation/tests/test_todo_completion.py
    TEST test_user_can_complete_todo
    TEST test_completed_todo_appears_in_completed_filter
    TEST test_completed_todo_is_hidden_from_active_filter
```

15.4 Expected V2 improvement

For "Complete an existing todo", V2 should rank test_user_can_complete_todo, TodoPage.complete_todo, and existing_todo rather than returning broad file snippets. The output becomes more precise and directly reusable for test design and code generation.

15.5 What V2 still will not solve

If the requirement says "mark an existing task as done" while the code uses "complete_todo", pure keyword ranking over AST symbols can still fail. V2 fixes granularity, not semantic synonym understanding. That remaining failure justifies V3.

15.6 Locators deliberately postponed

We discussed treating Playwright locators as first-class retrieval objects. They are important for the final system, but the first V2 implementation can focus on classes, methods, tests, and fixtures. Locator extraction can then be added to the structural index with support for both named locator attributes and inline get_by_role/get_by_label calls.

16. Repository Retrieval V3 - semantic RAG + exact symbols

V3 combines two complementary retrieval mechanisms. RAG answers semantic similarity questions; AST/symbol lookup answers exact code questions. They should not be forced into one mechanism.

16.1 Responsibilities

Mechanism	Best questions
Semantic RAG	What similar tests exist? How is similar behavior tested? Which Page Object methods and guidelines are relevant?
AST/symbol lookup	Does ComponentsPage.delete_component exist? What arguments does it accept? Where is fixture existing_component defined? Which import is needed?

16.2 Structure-aware RAG chunks

Embeddings should be created from semantic units rather than arbitrary token windows. A test function, Page Object method, fixture, class, or guideline section is a better chunk than lines 1-100.

```
Chunk content:
def complete_todo(self, title: str) -> None:
    self.todo_checkbox(title).check()

Metadata:
{
    "file": "pages/todo_page.py",
    "class": "TodoPage",
    "method": "complete_todo",
    "artifact_type": "method",
    "feature": "todo",
    "operation": "complete"
}
```

16.3 Example semantic recovery

```
Requirement:
"Mark an existing task as done"

Semantic retriever can associate:
task -> todo
done -> completed
mark as done -> complete

Likely retrieved context:
- test_user_can_complete_todo
- TodoPage.complete_todo
- existing_todo fixture
```

16.4 Exact verification before generation

Even when RAG finds a semantically relevant method, the generator should verify exact symbol facts before using them. This is how we prevent hallucinated framework APIs such as todo_page.mark_as_done() when the real method is todo_page.complete_todo().

17. Future Playwright MCP live exploration

Repository knowledge tells us how the framework used to interact with the application. Playwright MCP tells us what the live application exposes now. These are different truth sources and both are necessary.

17.1 MCP responsibilities

- Navigate the live application.
- Inspect accessible roles, names, labels, controls, and fields.
- Perform exploratory interactions.
- Validate assumptions from the requirement and repository.
- Observe results and UI state changes.
- Detect stale Page Object locators or changed behavior.

17.2 Repository vs live UI

```
Repository AST says:
get_by_role("button", name="Delete component")

Playwright MCP observes:
role=button, name="Remove"

Conclusion:
Existing automation locator may be stale.
Explore behavior before changing the test.
```

17.3 MCP is not the permanent regression suite

MCP actions are exploration and verification capabilities for the agent. Final tests should still be Python + Playwright + pytest so they remain reviewable, version-controlled, deterministic, and executable without the agent.

18. Future TestDesign and code generation

18.1 Why design before code

Generating code directly from a raw requirement encourages hallucinated setup, selectors, and framework APIs. The intended pipeline is Requirement -> Investigation -> structured TestDesign -> validation -> code generation.

18.2 Planned TestDesign

```
class TestDesign(BaseModel):
    title: str
    objective: str
    preconditions: list[str]
    test_data: list[str]
    steps: list[str]
    expected_results: list[str]
    required_pages: list[str]
    required_fixtures: list[str]
    relevant_existing_tests: list[str]
    risks: list[str]
    assumptions: list[str]
```

18.3 Context consumed by design_test

```
RequirementAnalysis
        +
RepositoryContext
        +
Exact symbols
        +
MCP exploration findings
        +
QA guidelines
        ->
TestDesign
```

18.4 Generation rules

- Reuse existing Page Object methods before creating new ones.
- Reuse fixtures and data builders before duplicating setup.
- Prefer Playwright expect assertions.
- Prefer get_by_role, get_by_label, get_by_placeholder, and intentional test IDs.
- Avoid hard-coded sleeps and wait_for_timeout for normal synchronization.
- Keep business behavior in tests and UI interaction details in Page Objects/components.
- Never invent a method, fixture, helper, or import without exact repository verification.

19. Future execution, result analysis, and repair

A generated test is not finished when the code compiles. It must run under pytest, and failures must be classified before any repair is attempted.

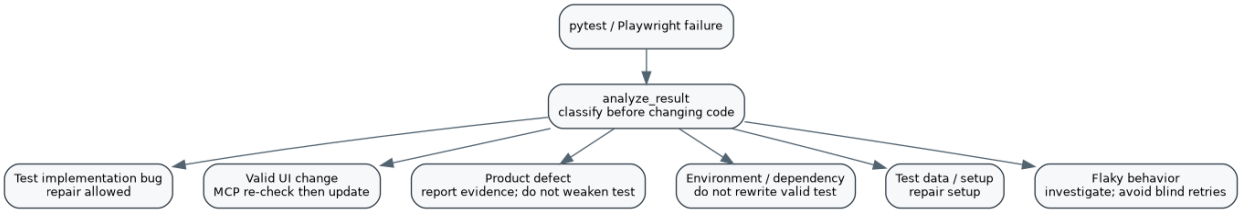


Figure 4. Failure classification precedes repair.

19.1 Failure taxonomy

Classification	Policy
Test implementation bug	Fix automation if evidence supports the change.
UI changed	Explore with MCP and update only if new behavior is valid.
Product defect	Preserve failing assertion/evidence and report; do not weaken test.
Environment problem	Do not rewrite valid automation to accommodate infrastructure failure.
Test-data issue	Repair setup/data source.
Flaky behavior	Investigate cause before adding retries.

19.2 Repair loop

```
run_test
    -> PASS -> review_test -> END
    -> FAIL -> analyze_result
                  |
                  +-> automation defect -> repair_test -> run_test
                  +-> product defect    -> report_findings -> END
                  +-> environment       -> report_findings -> END
```

Non-negotiable principle

Never change a failing test only to make it pass. A passing test is useless if it no longer validates the intended requirement.

20. Evaluation and observability strategy

20.1 Evaluate nodes independently

The requirement analyzer already has a deterministic dataset. Retrieval should receive its own cases, and later test design, tool use, generation, and failure classification should each have focused evaluation. Full-agent success alone is too coarse to diagnose regressions.

20.2 Retrieval evaluation dimensions

- Recall: did the relevant existing test/method/fixture appear?
- Precision: how much unrelated context was returned?
- Ranking: were the most reusable artifacts near the top?
- Grounding: did downstream generation use only retrieved/verified symbols?
- Context size: did retrieval stay compact enough for stable prompts?

20.3 Future RAGAS role

RAGAS can later complement deterministic checks with semantic metrics, especially once embeddings and repository RAG are introduced. Exact facts such as HTTP 409 or method existence should still be checked deterministically rather than delegated to another LLM.

20.4 Observability in graph state

Store concise structured results in QAState: analysis, retrieval references, design, generated code, execution status, failure classification, retry count, and final summary. Avoid storing large raw documents or entire repository files in state.

21. Security, configuration, and maintainability

21.1 Secrets

API keys belong in environment-backed settings and should remain SecretStr values. They should not be printed in logs, serialized into graph state, or committed to the repository.

21.2 Repository boundaries

TARGET_REPOSITORY makes the automation corpus an external dependency. The agent code should not contain knowledge specific to sample_automation; that directory is only the current target for development and evaluation.

21.3 Keep state small

State should contain selected context, not raw repositories. V2/V3 should return references and compact source chunks with metadata so downstream prompts remain bounded and observable.

21.4 Avoid premature multi-agent complexity

A single LangGraph workflow with explicit nodes provides enough agentic behavior for the current stage. Specialized Explorer, Test Designer, Reviewer, Failure Analyzer, or Supervisor agents should be added only when they provide measurable value beyond node-level prompts and tools.

22. Development roadmap

Phase	Status	Primary outcome
1. Python/Playwright pytest sample framework	Implemented	Real corpus to inspect and later generate against.
2. LangGraph state/workflow	Implemented	Controlled orchestration skeleton.
3. Requirement analyzer	Implemented + evaluated	Structured requirement understanding.
4. Repository retrieval V1	Implemented	File-level lexical context.
5. Repository retrieval V2	Next	AST classes/methods/tests/fixtures with symbol-level snippets.
6. Repository retrieval V3	Planned	Semantic RAG + exact AST lookup.
7. Playwright MCP exploration	Planned	Live application evidence.
8. TestDesign generation/validation	Planned	Grounded test plan before code.
9. Playwright test generation	Planned	Maintainable pytest/POM automation.
10. Test execution	Planned	pytest runner integration.
11. Failure classification	Planned	Evidence-based routing.
12. Limited self-repair	Planned	Repair automation defects only.
13. Evaluation/metrics	Continuous	Regression quality for each node and full workflow.
14. Multi-agent behavior	Later	Only if single-workflow architecture proves insufficient.

Immediate next implementation

Build V2 RepositoryIndexer using Python ast. Extract classes, methods, test functions, and pytest fixtures with path, parent, source, and line ranges. Keep RepositoryContextProvider unchanged, then run the three Todo requirements through V1 and V2 side-by-side.

23. Glossary

Term	Meaning in this project
Agentic workflow	A controlled graph of reasoning/tool nodes with explicit state and routing.
LangGraph	Orchestration layer managing state, nodes, edges, retries, and conditional transitions.
LangChain	Model/tool abstraction layer used by components such as ChatOpenAI.
Pydantic	Runtime-validated schemas for structured domain data and LLM output.
Protocol	Python structural interface used for dependency inversion and test fakes.
QAState	Typed shared state for one graph execution.
Partial state update	Dictionary returned by a node containing only fields that node changes.
Composition root	main.py location where concrete dependencies are constructed and injected.
RAG	Semantic retrieval of relevant repository/doc knowledge.
AST	Python syntax tree used for deterministic code structure and exact symbols.
Playwright MCP	Live browser exploration/verification tool for the agent.
Page Object Model	Automation architecture that keeps UI interaction details out of business-level tests.
Grounding	Ensuring generated decisions/code are supported by retrieved repository or live application evidence.

Appendix A. Consolidated current reference code

Reference status

The listings below consolidate the code structure developed in the project conversation into one readable reference. They are intended to match the current design, but your repository remains the authoritative source if a later local edit differs.

A.1 retrieval/models.py

```
from enum import StrEnum

from pydantic import BaseModel, Field


class RepositoryArtifactType(StrEnum):
    TEST = "test"
    PAGE_OBJECT = "page_object"
    FIXTURE = "fixture"
    GUIDELINE = "guideline"
    OTHER = "other"


class RetrievedContextItem(BaseModel):
    path: str
    artifact_type: RepositoryArtifactType
    score: float
    matched_terms: list[str] = Field(default_factory=list)
    snippet: str


class RepositoryContext(BaseModel):
    query_terms: list[str] = Field(default_factory=list)
    items: list[RetrievedContextItem] = Field(default_factory=list)
```

A.2 retrieval/provider.py

```
from typing import Protocol

from agentic_qa.models import RequirementAnalysis
from agentic_qa.retrieval.models import RepositoryContext


class RepositoryContextProvider(Protocol):
    def retrieve(
        self,
        analysis: RequirementAnalysis,
    ) -> RepositoryContext:
        ...
```

A.3 keyword_provider.py

```
import re
from collections.abc import Iterator
from pathlib import Path

from agentic_qa.models import RequirementAnalysis
from agentic_qa.retrieval.models import (
    RepositoryArtifactType,
    RepositoryContext,
    RetrievedContextItem,
)


class KeywordRepositoryContextProvider:
    SUPPORTED_SUFFIXES = {".py", ".md"}
```

```
STOP_WORDS = {
    "the", "and", "for", "with", "when", "that", "this",
    "should", "user", "users", "verify", "able", "from",
    "into", "after", "before", "then", "their", "they", "them",
}

IGNORED_DIRECTORIES = {
    ".git", ".venv", "__pycache__", ".pytest_cache",
    ".mypy_cache", ".ruff_cache",
}

def __init__(
    self,
    repository_path: Path,
    top_k: int = 8,
) -> None:
    self._repository_path = repository_path
    self._top_k = top_k

def retrieve(
    self,
    analysis: RequirementAnalysis,
) -> RepositoryContext:
    self._validate_repository()
    query_terms = self._build_query_terms(analysis)
    candidates: list[RetrievedContextItem] = []

    for path in self._iter_source_files():
        item = self._score_file(path=path, query_terms=query_terms)
        if item is not None:
            candidates.append(item)

    candidates.sort(key=lambda item: item.score, reverse=True)

    return RepositoryContext(
        query_terms=query_terms,
        items=candidates[: self._top_k],
    )

def _validate_repository(self) -> None:
    if not self._repository_path.exists():
        raise FileNotFoundError(
            f"Repository does not exist: {self._repository_path}"
        )
    if not self._repository_path.is_dir():
        raise NotADirectoryError(
            f"Repository path is not a directory: {self._repository_path}"
        )

def _build_query_terms(
    self,
    analysis: RequirementAnalysis,
) -> list[str]:
    values = [
        analysis.feature,
        analysis.operation,
        *analysis.conditions,
        *analysis.expected_outcomes,
    ]
    text = " ".join(values)
    words = re.findall(r"[A-Za-z0-9_]+", text.lower())
    terms = {
        word for word in words
        if len(word) >= 3 and word not in self.STOP_WORDS
    }
    return sorted(terms)

def _iter_source_files(self) -> Iterator[Path]:
    for path in self._repository_path.rglob("*"):
        if not path.is_file():
            continue
        if path.suffix not in self.SUPPORTED_SUFFIXES:
            continue
        if self._should_ignore(path):
            continue
        yield path
```

```
    def _should_ignore(self, path: Path) -> bool:
        return any(
            part in self.IGNORED_DIRECTORIES
            for part in path.parts
        )

    def _score_file(
        self,
        path: Path,
        query_terms: list[str],
    ) -> RetrievedContextItem | None:
        try:
            content = path.read_text(encoding="utf-8")
        except UnicodeDecodeError:
            return None

        relative_path = path.relative_to(self._repository_path)
        path_text = str(relative_path).lower()
        content_text = content.lower()
        score = 0.0
        matched_terms: list[str] = []

        for term in query_terms:
            term_score = 0.0
            if term in path_text:
                term_score += 5.0
            occurrences = content_text.count(term)
            if occurrences > 0:
                term_score += min(float(occurrences), 3.0)
            if term_score > 0:
                matched_terms.append(term)
                score += term_score

        if score == 0:
            return None

        return RetrievedContextItem(
            path=str(relative_path),
            artifact_type=self._classify_file(relative_path),
            score=score,
            matched_terms=matched_terms,
            snippet=self._extract_snippet(
                content=content,
                matched_terms=matched_terms,
            ),
        )

    def _classify_file(self, path: Path) -> RepositoryArtifactType:
        if path.name == "conftest.py":
            return RepositoryArtifactType.FIXTURE
        if "tests" in path.parts and path.name.startswith("test_"):
            return RepositoryArtifactType.TEST
        if "pages" in path.parts:
            return RepositoryArtifactType.PAGE_OBJECT
        if path.suffix == ".md":
            return RepositoryArtifactType.GUIDELINE
        return RepositoryArtifactType.OTHER

    def _extract_snippet(
        self,
        content: str,
        matched_terms: list[str],
        context_lines: int = 5,
    ) -> str:
        lines = content.splitlines()
        for index, line in enumerate(lines):
            line_lower = line.lower()
            if any(term in line_lower for term in matched_terms):
                start = max(0, index - context_lines)
                end = min(len(lines), index + context_lines + 1)
                return "\n".join(lines[start:end])
        return "\n".join(lines[: 2 * context_lines])
```

A.4 graph/nodes.py - current conceptual form

```
from typing import Any, Protocol

from agentic_qa.graph.state import QAState
from agentic_qa.models import (
    InteractionSurface,
    RequirementAnalysis,
    ResultAnalysis,
    TestDesign,
    TestExecutionResult,
)
from agentic_qa.requirements.analyzer import RequirementAnalyzer
from agentic_qa.retrieval.models import RepositoryContext
from agentic_qa.retrieval.provider import RepositoryContextProvider


class QANode(Protocol):
    def __call__(self, state: QAState) -> Any:
        ...


def create_analyze_requirement_node(
    analyzer: RequirementAnalyzer,
) -> QANode:
    def analyze_requirement(
        state: QAState,
    ) -> dict[str, RequirementAnalysis]:
        analysis = analyzer.analyze(state["requirement"])
        return {"requirement_analysis": analysis}

    return analyze_requirement


def create_retrieve_context_node(
    provider: RepositoryContextProvider,
) -> QANode:
    def retrieve_context(
        state: QAState,
    ) -> dict[str, RepositoryContext]:
        context = provider.retrieve(state["requirement_analysis"])
        return {"repository_context": context}

    return retrieve_context


def explore_ui(state: QAState) -> dict[str, list[str]]:
    analysis = state["requirement_analysis"]
    if analysis.interaction_surface != InteractionSurface.UI:
        return {"exploration_findings": []}
    return {
        "exploration_findings": [
            "UI exploration is not connected yet."
        ]
    }
```

A.5 main.py - composition root pattern

```
def create_workflow() -> CompiledStateGraph[
    QAState, None, QAState, QAState
]:
    if settings.openai_api_key is None:
        raise RuntimeError("OPENAI_API_KEY is not configured.")

    if not settings.llm_model:
        raise RuntimeError("LLM_MODEL is not configured.")

    model = ChatOpenAI(
        model=settings.llm_model,
        api_key=settings.openai_api_key,
    )

    requirement_analyzer = LLMRequirementAnalyzer(model=model)
```

```
    repository_context_provider = KeywordRepositoryContextProvider(
        repository_path=settings.target_repository,
        top_k=8,
    )

    return build_workflow(
        requirement_analyzer=requirement_analyzer,
        repository_context_provider=repository_context_provider,
    )
```

Appendix B. Debugging lessons learned

B.1 LangGraph InvalidUpdateError from node factory

A factory function itself was accidentally passed to add_node instead of calling the factory. LangGraph executed the factory as a node and received a function object rather than a state-update dictionary. The fix was to call create_analyze_requirement_node(requirement_analyzer) when registering the node.

B.2 ChatOpenAI api_key mypy error

Calling get_secret_value() produced str while the installed langchain-openai typing expected SecretStr-compatible input. Passing settings.openai_api_key directly resolved the mismatch and preserved secret handling.

B.3 create_workflow missing return annotation

With disallow_untyped_defs enabled, functions require explicit return annotations. We used CompiledStateGraph[QAState, None, QAState, QAState] rather than Any.

B.4 add_node overload mismatch

Annotating a node factory as Callable[[QAState], dict[str, Any]] still did not satisfy LangGraph add_node overload typing. A local QANode Protocol with __call__(state: QAState) -> Any gave a stable structural boundary without importing private LangGraph types.

B.5 stale needs_ui_exploration field

After the analyzer schema evolved, explore_ui and FakeRequirementAnalyzer still referenced needs_ui_exploration. The field was replaced by interaction_surface. This was a reminder to treat domain model changes as cross-cutting changes that must update production code and test doubles together.

B.6 brittle fake expectations

Tests expected feature="components" while FakeRequirementAnalyzer had been updated to feature="Component management". A better node test compares the returned object with analyzer.result rather than duplicating fake internals in the assertion.

B.7 V1 retrieved the right file for the wrong reason

The semantic-mismatch requirement "mark an existing task as done" ranked the completion test first mostly because both contained existing. This is a classic retrieval-quality trap: correct top result does not necessarily mean the retriever understood the query. Evaluation must inspect match reasons, not only top-1 identity.

Closing mental model

The easiest way to understand the project is to think of each technology as owning one responsibility. The system becomes maintainable when those responsibilities stay separate.

```
LLM
    -> understand / reason / generate

LangGraph
    -> orchestrate state and control flow

Pydantic
    -> validate structured contracts

Repository retrieval
    -> provide relevant existing knowledge

AST
    -> provide exact code structure

Playwright MCP
    -> provide live application evidence

pytest + Playwright
    -> execute permanent regression automation

Git
    -> review and version the produced code
```

Next learning milestone

Implement V2 AST indexing and compare V1 vs V2 on the same three real Todo requirements. Do not add embeddings until we have measured which failures remain after structure-aware retrieval.

Evaluation documentation revision — 18 October 2026

Documentation update | 18 October 2026

STATUS: The original handbook chapters are preserved as historical milestones. This revision supersedes older “current” claims about 7-case retrieval baselines and the V2 retriever being planned. All results below are from the two user-provided LangSmith CSV exports; no new LangSmith evaluation was executed during this update.

Current workflow and scope

Requirement -> analyze_requirement -> retrieve_context -> inspect_symbols (planned/partly independent) -> explore_ui (MCP integration planned) -> design_test -> generate_test -> run_test -> analyze_result -> review/repair. The actively evaluated retrieval strategies are V1.2 token-aware file retrieval and V2.0 AST symbol retrieval followed by unique-file projection. V2.1 dependency expansion is proposed, not implemented.

Repository retrieval datasets

repository-retrieval-realworld-v1: frozen 7-case benchmark (4 auth, 3 article). repository-retrieval-realworld-v2: 10-case benchmark (4 auth, 3 article, 3 comment), run on the expanded sample_automation corpus. Historic v1 experiments must be retained; comparisons between algorithm versions should use the same dataset version and repository revision.

V2.0 retrieval implementation

PythonAstSymbolIndexer produces RepositorySymbolIndex. SymbolRepositoryContextProvider ranks individual symbols using exact token matches in names, parameters, parent/path and source. project_symbols_to_repository_context deduplicates ranked symbols into a RepositoryContext of unique file paths (top 8). V1.2 remains token-aware whole-file keyword retrieval. Neither retriever currently performs relationship-driven fixture/import dependency expansion.

LangSmith evaluation v2 | measured macro averages

Metric | V1.2 | V2.0 | Difference (percentage points): precision_at_3: 46.7% -> 50.0% (+3.3 pp); recall_at_3: 33.7% -> 35.3% (+1.6 pp); recall_at_5: 45.5% -> 49.8% (+4.3 pp); top1_relevant: 80.0% -> 100.0% (+20.0 pp). All 10 cases completed in both exports. Top1 improves markedly; recall remains moderate.

Per-case Recall@5

Edit article: V1.2 60% / V2.0 40%; Delete own comment: V1.2 17% / V2.0 33%; View existing comment: V1.2 20% / V2.0 20%; Delete article: V1.2 40% / V2.0 20%; Register a new user: V1.2 50% / V2.0 50%; Sign in with registered credentials: V1.2 25% / V2.0 75%; Attempt to sign up with a valid email and password while leaving the username empty: V1.2 100% / V2.0 67%; Create comment: V1.2 17% / V2.0 33%; Create article: V1.2 60% / V2.0 60%; Sign in with a valid registered email and an invalid password: V1.2 67% / V2.0 100%

Interpretation and limitation

V2.0 improves primary-artifact identification (Top1: 80% to 100%) with limited improvement in context completeness (Recall@5: 45.5% to 49.8%). Ranked results are often dominated by similar test files instead of fixtures, API client, Page Objects, and data builders. In Delete Comment, V2.0 returns only two of six required artifacts by rank 8: the test and comments fixture; the Page Object, Comment component, API client, and builder are absent. This supports evaluating dependency-aware expansion, not blindly changing weights. A 10-case benchmark is small and includes existing tests whose names echo requirements; results do not establish generalization to unseen behaviors.

V2.1 plan and evaluation contract

PROPOSED, not implemented: start from top ranked tests/symbols; inspect pytest fixture parameters, local imports and Page Object/component references; resolve exact symbol definitions with AST; add limited one-hop related artifacts; project to deduplicated files. Evaluate V2.1 against frozen v2 dataset and same corpus; retain Top1 while improving Recall@3/5/8, monitor precision and failure cases. Do not modify V1.2 or V2.0 during the comparison.

Reproducibility and snapshot discrepancy

Current user-reported LangSmith runs targeted repository-retrieval-realworld-v2. The attached agentic-qa(2).zip snapshot still contains scripts pointing to repository-retrieval-realworld-v1 and an empty comments.py evaluation list. Treat this as a snapshot mismatch: the successful v2 experiments and CSV exports evidence newer local changes not included in that archive. Synchronize a committed repository state before any future rerun; do not claim the archive itself contains the 10-case dataset source.

Quality safeguards

Run ruff, mypy and pytest; keep fixture setup/cleanup API-backed and browser behavior UI-backed. In single-script mypy invocations, ensure the local src/ package is discoverable and typed (py.typed if distributing as PEP 561 typed package). Do not confuse mypy packaging problems with retrieval evaluation regressions. Preserve failing test evidence before any repair.

Evidence

Sources: 10-row CSV files realworld-dataset-v2-retrieval-v1.2-token-aware-48b9dd6a.csv and realworld-datadset-v2-retrieval-v2.0-symbol-token-aware-94cb88d6.csv; repository snapshot agentic-qa(2).zip; historical docs/ handbooks. Numbers are macro averages of exported feedback scores.